

Empirical Evaluation of Small Language Models for Intent-Driven Slice Lifecycle Management in 6G Core Networks

Teerapong Somsri, [Co-authors TBD]

Rajabhat Ratchanakarin University (RBRU)

Abstract

The deployment of intent-based networking (IBN) in 6G core networks requires lightweight, efficient language models capable of translating natural language intents into network orchestration commands. This paper presents a comprehensive empirical evaluation of small language models (SLMs) for intent-driven slice lifecycle management, examining both cross-family performance (Llama, Qwen, Gemma, GLM at ~8B parameters) and the impact of model size scaling (0.5B-7B) with quantization (Q2_K, Q4_K_M, Q8_0). Using a dataset of 200 intents spanning provisioning, scaling, and conflict resolution domains, we evaluate models across seven dimensions with per-intent CPU, memory, and energy monitoring. Our results reveal that Qwen2.5-7B achieves the highest overall accuracy (29.8% exact match, 43.5% action accuracy), while Qwen2.5-3B-Q4_K_M offers the best cost-effectiveness. A key finding is that CPU utilization scales with model size (69% at 0.5B to 94% at 7B), indicating that memory bandwidth — not compute — is the primary bottleneck for larger models. We also find that Q4_K_M quantization reduces memory by ~50% with less than 1pp accuracy loss, and identify a "knee" at 3B beyond which accuracy gains diminish while costs grow linearly. These findings provide practical deployment guidelines for 6G edge infrastructure.

Keywords: Small Language Models, 6G Networks, Intent-Based Networking, Network Slicing, Model Quantization, Edge AI, Resource Efficiency

1. Introduction

Sixth-generation (6G) networks must support diverse use cases including enhanced mobile broadband (eMBB), ultra-reliable low-latency communications (URLLC), and massive machine-type communications (mMTC). Network slicing is a cornerstone technology for meeting these heterogeneous requirements [1,2]. Intent-based networking (IBN) allows operators to express high-level goals in natural language, automatically translated into low-level configurations [3]. Large language models (LLMs) have shown remarkable capabilities for this translation task [4,5], but their computational requirements are often incompatible with edge infrastructure where power budgets may be as low as 10-50W [6].

Small language models (SLMs) with 0.5-8B parameters, combined with quantization (2-8 bits), offer a potential solution — but their performance on domain-specific networking tasks remains underexplored. This paper addresses four research questions:

- RQ1: How do different SLM architectures (Llama, Qwen, Gemma, GLM) compare in intent translation accuracy?
- RQ2: What is the relationship between model size (0.5B-7B) and intent translation performance?
- RQ3: How does quantization (Q2_K, Q4_K_M, Q8_0) affect the accuracy-efficiency trade-off?
- RQ4: What are the resource consumption characteristics (CPU, memory, energy) of SLM inference on CPU-only hardware?

Our contributions include: (1) the first comprehensive comparison of open-weight SLMs for 6G intent translation with per-intent resource monitoring, (2) a detailed Pareto frontier analysis across model sizes and quantization levels with 95% confidence intervals, (3) the novel finding that CPU utilization scales with model size, revealing the memory-bandwidth bottleneck, and (4) practical deployment recommendations for resource-constrained 6G edge infrastructure.

2. Methodology

2.1 Dataset & Models

We constructed a dataset of 200 natural language intents spanning three network slice lifecycle domains: Slicing Provisioning (70), Scaling Request (70), and Conflict Resolution (60). Each intent contains an input_text and ground_truth JSON with expected action, slice type, and parameters. Phase 1 evaluates four open-weight SLM families at ~8B with Q4_K_M quantization: Llama 3.1-8B [10], Qwen2.5-7B [11], Gemma 2-9B [12], and GLM-4-9B [13]. DeepSeek-R1-8B was excluded due to incompatibility with CPU-only inference — its reasoning architecture caused infinite chain-of-thought loops [20]. Phase 2 evaluates Qwen2.5 (top Phase 1 performer) across 4 sizes $\times$ 3 quantization levels = 12 configurations.

2.2 Metrics & Statistical Framework

Seven metrics: Format Validity (valid JSON), Action Accuracy (correct action), Avg Match% (mean percentage of ground-truth key-value pairs matched), Inference Time (seconds), Throughput (tokens/s), Memory Footprint (peak RSS in MB via psutil), and Energy Consumption (J) calculated as $\text{measured_CPU_utilization} \times \text{TDP_135W} \times \text{inference_time}$. All continuous metrics report mean, standard deviation, and 95% confidence interval across $n=200$ intents using t-distribution ($df=199$, $t \approx 1.972$).

2.3 Hardware

All experiments run on identical Proxmox VE 8.2 KVM virtual machines (16 vCPUs Intel Xeon E5-2690 v4, 32GB RAM, 80GB SSD). Software: Ubuntu 24.04, Ollama 0.5.x (llama.cpp),

Python 3.12, psutil 6.x. Each configuration includes a 2-intent warmup for consistent KV-cache state.

3. Phase 1: Cross-Family Model Comparison

Table 1 presents cross-family results at ~8B with Q4_K_M quantization.

Model	Format %	Action %	Avg Match %	Time (s)
Llama 3.1 8B	100.0	36.5	26.0	19.11
Qwen2.5 7B	100.0	43.5	29.8	18.68
Gemma 2 9B	98.5	45.5	29.3	39.79
GLM-4 9B	100.0	29.0	24.6	22.88

Table 1: Phase 1 cross-family results (~8B, Q4_K_M, n=200).

Qwen2.5-7B achieves the best overall balance (29.8% match, 43.5% action, 18.68s). Gemma 2-9B has highest action accuracy (45.5%) but at 2.1× latency (39.79s). GLM-4-9B underperforms significantly (29.0% action). The 9B models show no proportional gains over 7-8B, indicating architecture quality outweighs raw parameter count. Qwen2.5 is selected for Phase 2.

4. Phase 2: Size Scaling & Quantization

4.1 Complete Results with 95% CI

Config	Match%	95% CI	CPU%	95% CI	Time(s)	95% CI	Energy(J)	95% CI	Action%	Mem(MB)
0.5B Q2 K	18.3	[14.9,21.8]	68.8	[67.0,70.6]	4.3	[3.9,4.7]	436	[384,487]	24.0	2605
0.5B Q4 K M	18.5	[15.0,22.0]	69.1	[67.2,71.1]	4.7	[4.3,5.0]	477	[429,525]	32.5	575
0.5B Q8 0	19.4	[16.0,22.8]	71.6	[70.0,73.3]	4.8	[4.5,5.2]	505	[454,555]	26.0	2166
1.5B Q2 K	22.1	[18.4,25.7]	76.9	[75.3,78.5]	7.8	[5.5,10.1]	897	[592,1202]	40.0	1372
1.5B Q4 K M	25.8	[21.9,29.7]	80.6	[79.5,81.8]	7.5	[7.0,8.1]	850	[780,920]	41.0	1720
1.5B Q8 0	25.7	[21.6,29.7]	83.1	[82.2,84.1]	5.3	[5.0,5.7]	611	[566,656]	40.0	2480
3B Q2 K	24.1	[20.3,28.0]	85.8	[85.1,86.4]	9.5	[8.9,10.1]	1117	[1037,1198]	42.5	2453
3B Q4 K M	27.3	[23.2,31.4]	86.1	[85.3,87.0]	9.4	[8.8,10.1]	1122	[1039,1205]	45.5	3323
3B Q8 0	26.8	[22.7,30.9]	88.4	[87.8,89.1]	7.7	[7.1,8.2]	931	[857,1004]	44.0	5225
7B Q2 K	29.0	[24.9,33.0]	91.2	[90.8,91.6]	13.8	[12.7,14.8]	1719	[1580,1857]	42.0	8149
7B Q4 K M	29.8	[25.8,33.9]	91.9	[91.4,92.3]	15.0	[13.9,16.1]	1884	[1734,2034]	43.5	4922
7B Q8 0	30.5	[26.3,34.6]	94.1	[93.8,94.5]	20.4	[18.9,21.8]	2613	[2420,2806]	44.0	11429

Table 2: Phase 2 complete results with 95% CI — Qwen2.5 across 4 sizes × 3 Q levels (n=200).

CPU% measured via psutil, Energy from actual CPU utilization × TDP (135W) × time.

4.2 Size Scaling Analysis

- 0.5B → 1.5B: Avg match improves from 18.7% to 24.5% (+5.8pp, +31% relative). Largest single-step gain — 0.5B models lack capacity for structured JSON.
- 1.5B → 3B: Avg match improves from 24.5% to 26.1% (+1.6pp, +6.5% relative). Marginal gain drops significantly.
- 3B → 7B: Avg match improves from 26.1% to 29.8% (+3.7pp, +14% relative), but at 67% more time (8.8s → 16.4s avg) and 80% more energy (972J → 2,072J avg). Pareto "knee" at 3B.

4.3 Quantization Impact

- Q8_0 → Q4_K_M: Avg match loss 0.2pp, memory reduced ~50%. Q4_K_M is the optimal trade-off.
- Q4_K_M → Q2_K: Avg match loss 1.6pp, energy savings 20-30%. 7B-Q2_K (29.0% match, 13.8s, 1,719J) outperforms 3B-Q8_0 (26.8%) with only 85% more energy.
- Quantization sensitivity: 0.5B least sensitive (± 1.1 pp), 1.5B most sensitive (3.7pp Q2_K → Q4_K_M).

4.4 CPU Utilization: The Memory Bandwidth Bottleneck

A novel finding of our study is that CPU utilization is not constant across model sizes — it scales from 68.8% (0.5B-Q2_K) to 94.1% (7B-Q8_0). This reveals that inference on CPU-only hardware is memory-bandwidth-limited: smaller models with smaller weight matrices do not fully saturate the memory bus, leaving CPU cores idle. As model size increases, weight transfer dominates, pushing utilization toward saturation. This has important implications for energy estimation — assuming a flat 95% CPU utilization overestimates energy for small models by 15-25% (e.g., 0.5B-Q4_K_M: 477J actual vs 599J estimated). Table 3 presents the full CPU utilization matrix.

Size	Q2_K CPU%	Q4_K_M CPU%	Q8_0 CPU%
0.5B	68.8%	69.1%	71.6%
1.5B	76.9%	80.6%	83.1%
3B	85.8%	86.1%	88.4%
7B	91.2%	91.9%	94.1%

Table 3: CPU utilization by model size and quantization (system-wide psutil measurement).

5. Resource Consumption Analysis

5.1 Energy Efficiency Ranking

Rank	Config	Match%	Energy(J)	CPU%	Match/ J×100	Mem(MB)
1	7B Q8 0	30.5	2613	94.1%	1.167	11429
2	7B Q4 K M	29.8	1884	91.9%	1.582	4922

3	7B Q2 K	29.0	1719	91.2%	1.687	8149
4	3B Q2 K	24.1	1117	85.8%	2.157	2453
5	3B Q4 K M	27.3	1122	86.1%	2.433	3323
6	1.5B Q2 K	22.1	897	76.9%	2.464	1372
7	3B Q8 0	26.8	931	88.4%	2.879	5225
8	1.5B Q4 K M	25.8	850	80.6%	3.034	1720
9	0.5B Q8 0	19.4	505	71.6%	3.843	2166
10	0.5B Q4 K M	18.5	477	69.1%	3.878	575
11	0.5B Q2 K	18.3	436	68.8%	4.201	2605
12	1.5B Q8 0	25.7	611	83.1%	4.206	2480

Table 4: Energy efficiency ranking — sorted by Match%/Joule (higher is better).

0.5B-Q2_K achieves highest efficiency (0.042 Match/J) due to low CPU utilization (68.8%). 7B-Q8_0 is least efficient (0.012 Match/J) — 3.5× less efficient per unit accuracy. The 1.5B-3B Q4_K_M range represents the sweet spot: competitive accuracy with practical energy budgets.

5.2 Memory Footprint

Peak RSS ranges from 575 MB (0.5B-Q4_K_M) to 11,429 MB (7B-Q8_0). 7B-Q8_0 exceeds the 8 GB budget typical of edge accelerators, making 7B-Q4_K_M (4,922 MB) or 7B-Q2_K (8,149 MB) more practical. Memory scales ~linearly with model size within each quantization level.

6. Domain-Level Analysis

Table 5 decomposes performance across three lifecycle domains (Q4_K_M configurations).

Size	Provisioning Match%/Act%	Scaling Match%/Act%	Conflict Match%/Act%	Overall
0.5B	35.8%/76.1%	15.6%/14.3%	2.5%/8.3%	18.5%/32.5%
1.5B	38.5%/71.4%	21.8%/14.3%	3.2%/8.3%	25.8%/41.0%
3B	40.2%/74.3%	22.1%/15.7%	3.8%/10.0%	27.3%/45.5%
7B	41.5%/75.7%	23.5%/14.3%	4.1%/11.7%	29.8%/43.5%

Table 5: Domain-level performance (Q4_K_M). Provisioning is well-handled; conflict resolution challenges all SLMs.

Provisioning achieves 35-42% match and 70-76% action accuracy due to well-defined schemas. Scaling shows 15-24% match — ambiguity in scaling magnitude limits performance. Conflict resolution is the hardest (2-4% match), requiring multi-constraint optimization beyond current SLM capabilities.

7. Discussion

7.1 Deployment Recommendations

- Latency-critical edge (<10s): Qwen2.5-3B-Q4_K_M — 27.3% match, 9.4s, 1,122J, 3.3GB. 91% of 7B accuracy at 60% cost.
- Accuracy-prioritized: Qwen2.5-7B-Q8_0 — 30.5% match, 20.4s, 2,613J. Upper bound for CPU-only SLM inference.
- Ultra-light edge (<2GB): Qwen2.5-1.5B-Q4_K_M — 25.8% match, 7.5s, 850J, 1.7GB. Competitive accuracy at minimal resources.
- Quantization: Q4_K_M recommended for all deployments. Q2_K acceptable when memory-constrained but avoid for 1.5B models.
- Energy budgeting: Use actual CPU% measurements, not flat assumptions — small models consume 15-25% less energy than TDP-based estimates.

7.2 Limitations

- CPU-only inference: Results specific to CPU. GPU would change latency-energy trade-off.
- Energy estimation: TDP-based ($\pm 15\%$). Direct RAPL measurement unavailable on VM infrastructure.
- Dataset scope: 200 intents may not capture full production diversity.
- Single architecture (Phase 2): Findings specific to Qwen2.5. Other architectures may differ.
- DeepSeek-R1 excluded: Reasoning models incompatible with CPU inference. GPU evaluation warranted.

7.3 Future Work

- Multi-agent orchestration with domain-specialized SLMs.
- Instruction-tuning on 6G data, particularly for conflict resolution.
- GPU benchmarking to establish performance bounds.
- Online learning through operator feedback.
- Integration with formal verification for safety guarantees.

8. Conclusion

This paper presented a comprehensive empirical evaluation of small language models for intent-driven slice management in 6G core networks, with per-intent CPU, memory, and energy monitoring across 16 model configurations on identical CPU-only hardware. Key findings include:

- Qwen2.5-7B achieves the highest accuracy (29.8% match, 43.5% action) among open-weight SLMs.
- CPU utilization scales with model size (69% $\rightarrow$ 94%), revealing memory bandwidth as the primary bottleneck.
- Diminishing returns beyond 3B — 3B achieves 91% of 7B accuracy at 60% cost.

- Q4_K_M provides optimal trade-off (≤ 1 pp loss, $\sim 50\%$ memory reduction).
- Q2_K is a viable option for memory-constrained scenarios with acceptable accuracy loss.
- 1.5B-3B Q4_K_M is the deployment sweet spot for 6G edge infrastructure.

References

- [1] I. Afolabi et al., "Network Slicing and Softwarization: A Survey," IEEE Commun. Surveys Tuts., vol. 20, no. 3, 2018.
- [2] K. Abbas et al., "Intent-Based Networking for 6G," IEEE Commun. Mag., vol. 62, no. 1, 2024.
- [3] ETSI GS ZSM 002, "Zero-touch Network and Service Management; Reference Architecture," 2019.
- [4] J. Wei et al., "Chain-of-Thought Prompting," NeurIPS, 2022.
- [5] A. Maatouk et al., "LLMs for Telecom," IEEE Commun. Mag., vol. 62, no. 3, 2024.
- [6] M. G. S. Murshed et al., "ML at the Network Edge," ACM Comput. Surv., vol. 54, no. 8, 2022.
- [7] ETSI GS ZSM 001, "ZSM Requirements," 2019.
- [8] 3GPP TS 28.530, "Management and Orchestration," 2023.
- [9] M. Mechtri et al., "Intent-Based Networking Survey," IEEE Access, vol. 11, 2023.
- [10] Meta AI, "Llama 3.1: Open Foundation Models," 2024.
- [11] Alibaba Cloud, "Qwen2.5 Technical Report," 2024.
- [12] Google DeepMind, "Gemma 2 Technical Report," 2024.
- [13] Zhipu AI, "GLM-4 Technical Report," 2024.
- [14] G. Gerganov, "GGUF Format," github.com/ggerganov/ggml, 2024.
- [15] T. Dettmers et al., "QLoRA," NeurIPS, 2023.
- [16] D. Patterson et al., "Carbon Emissions and Neural Network Training," arXiv:2104.10350, 2021.
- [17] R. Schwartz et al., "Green AI," Commun. ACM, vol. 63, no. 12, 2020.
- [18] S. Han et al., "EIE: Efficient Inference Engine," ISCA, 2016.
- [19] T. Dettmers et al., "LLM.int8()," NeurIPS, 2022.
- [20] T. Somsri et al., "SLM Agent Eval for 6G," GitHub, 2025.